

第4章 Verilog HDL语言基础（续）

4.5.4 模块例化

- 模块例化（**Module Instantiation**）是指在当前的设计中调用已经设计完成的其他模块的过程；
- 被调用的模块叫做实例（**Instance**）、实体或元件；
- 可重复利用已有成果，提高设计效率；
- 模块例化是结构化描述方式的基础。

4.5.4.1 模块例化语句

采用模块例化的步骤：

- 被调用的模块已经设计完成，可以是自己设计的，也可以是Verilog语言系统提供的。
- 在当前设计中，采用例化语句调用元件。

编写半加器程序，把它作为元件

```
module  H_Adder (A, B, C, R);
    input  A, B;
    output  R, C;

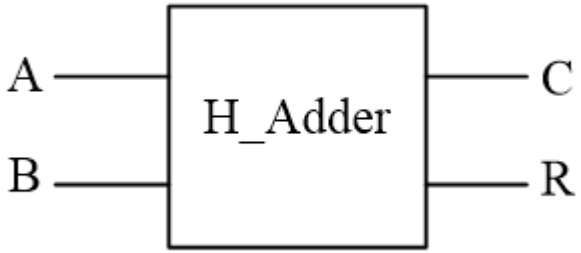
    assign  R = A ^ B;
    assign  C = A & B;
endmodule
```

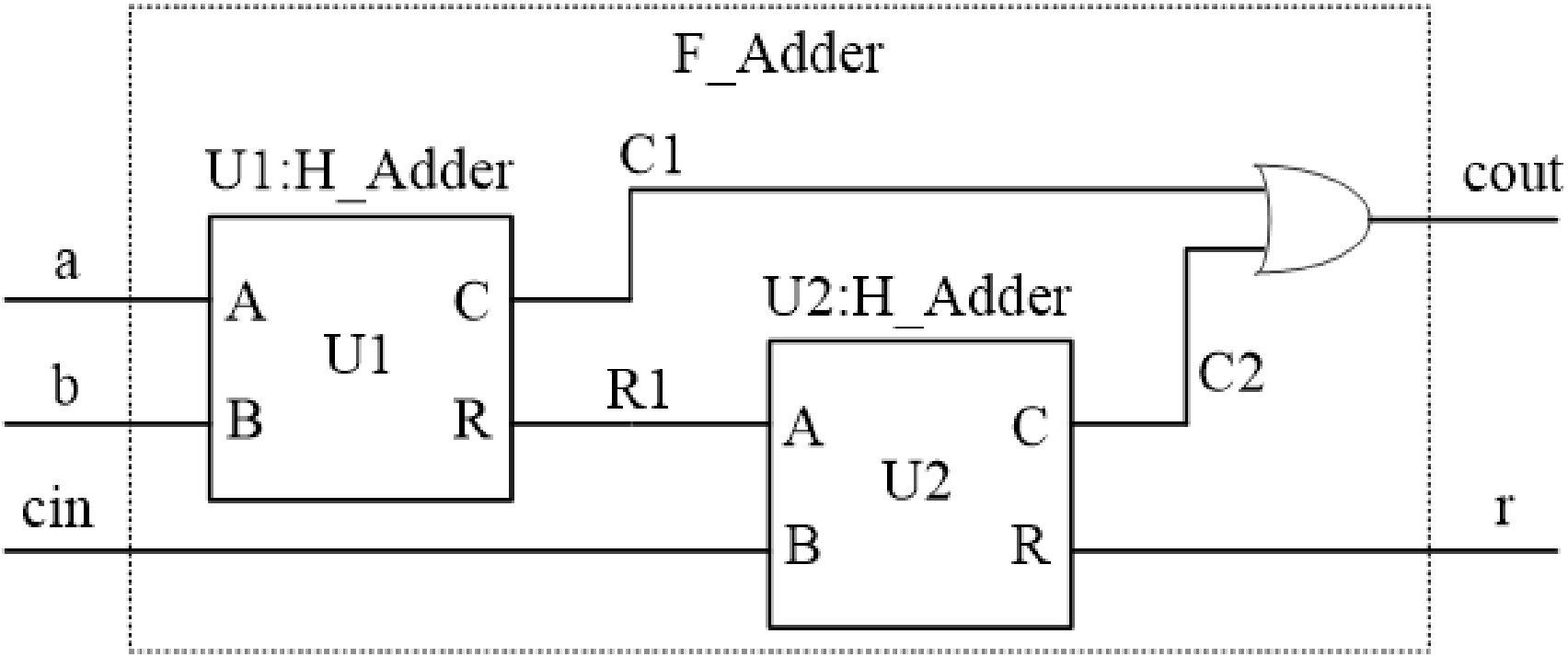
```
    A
+ B
-----
C  R
```

编写全加器程序，调用半加器H_Adder

```
module  F_Adder (a, b, cin, r, cout);
    input  a, b, cin;
    output  r, cout;
    wire  R1, C1, C2;

    H_Adder  U1 (.A(a),  .B(b),  .C(C1),  .R(R1));
    H_Adder  U2 (R1,  cin,  r,  C2);
    assign cout = C1 | C2;
endmodule
```


A
B
H_Adder
C
R


F_Adder
U1:H_Adder
C1
cout
a
b
cin
A
B
U1
C
R
R1
U2:H_Adder
A
B
U2
C
R
C2
r

全加器结构图

模块例化语句语法格式：

//端口名关联法

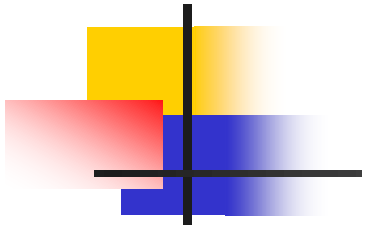
元件名 例化名 （.元件端口名1（外接端口名1），
.元件端口名2（外接端口名2）， ……）；

H_Adder U1 (.A(a), .B(b), .C(C1), .R(R1));

// 位置关联法

元件名 例化名 （端口名1，端口名2，…，端口名n）；

H_Adder U2 (R1, cin, r, C2);



模块例化语句

- 端口名关联法即要列出元件端口，又要列出当前模块端口或线网类型变量，它们之间有明确的对应关系，书写时顺序可变；
- 位置关联法只列出当前模块的端口或线网变量，通过位置隐含地说明连接关系，书写顺序不能变。
- 调用元件时，元件的某些端口如果不需要连接，可以写成诸如.A( )的形式，括号内是空的，表示该端口悬空。
- 悬空的端口如果是输入端口，其值为高阻态z；如果是输出端口，表示该端口不使用。

半加器和全加器写在一个程序里

F_Adder.v

```
1  module F_Adder(a, b, cin, r, cout);
2     input a, b, cin;
3     output r, cout;
4     wire R1, C1, C2;
5     H_Adder U1(.A(a), .R(R1), .B(b), .C(C1));
6     H_Adder U2(R1, cin, r, C2);
7     assign cout = C1 | C2;
8  endmodule
9
10 module H_Adder(A, B, R, C);
11    input A, B;
12    output R, C;
13    assign R = A ^ B;
14    assign C = A & B;
15 endmodule
16
```

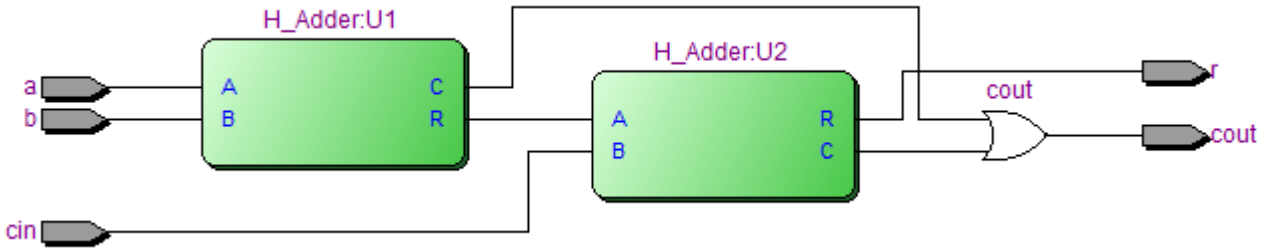
半加器和全加器各写一个程序

H_Adder.v

```
1  module H_Adder(A, B, R, C);
2     input A, B;
3     output R, C;
4     assign R = A ^ B;
5     assign C = A & B;
6  endmodule
7
```

F_Adder.v

```
1  module F_Adder(a, b, cin, r, cout);
2     input a, b, cin;
3     output r, cout;
4     wire R1, C1, C2;
5     H_Adder U1(.A(a), .R(R1), .B(b), .C(C1));
6     H_Adder U2(R1, cin, r, C2);
7     assign cout = C1 | C2;
8  endmodule
```



4.5.4.2参数化模块例化语句

- 如果一个元件功能固定，其重复使用的可能性就小一些；
- 在设计元件时可以加上一些参数，形成参数化元件，增加元件的重复利用率；
- 模块例化时，可以通过带参数的例化语句或参数定义语句（defparam）来实现不同功能的元件。
- 例如，一个参数化4×4乘法器就可以通过参数传递，作为8×8乘法器、 16×16乘法器来使用。

【例 5-26】带参数的 4×4 乘法器

```
module MULTIPLER4 (X, Y, R);
 parameter W=4;
 input [W:1] X, Y;
 output [2*W:1] R;
 reg [2*W:1] R;
 integer i;
 always @(X or Y)
   begin
      R = 0;
      for (i=1; i<=W; i=i+1)
          if (Y[i])
             R = R + (X<<(i-1));
   end
endmodule
```

参数也可以写在模块名之后

```
module MULTIPLER4 #( parameter W=4; ) (X, Y, R);
 input [W:1] X, Y;
 output [2*W:1] R;
 reg [2*W:1] R;
 integer i;
 always @(X or Y)
   begin
     R = 0;
     for (i=1; i<=W; i=i+1)
         if (Y[i])
            R = R + (X<<(i-1));
   end
endmodule
```

参数化元件调用

```
module MULTIPLER8_1 (X1,  Y1,  R1);
input  [7:0]  X1,   Y1;
output  [15:0]  R1;
    MULTIPLER4  #(.W(8))  L1 (.X(X1),  .Y(Y1),  .R(R1));
endmodule
```

或者

```
module MULTIPLER8_1 (X1,  Y1,  R1);
input  [7:0]  X1,   Y1;
output  [15:0]  R1;
    defparam  L1.W = 8;
    MULTIPLER4  L1 (.X(X1),  .Y(Y1),  .R(R1));
endmodule
```

8×8乘法器的端口名可以与4×4乘法器端口名一样，例如：

```
module MULTIPLER8_1 (X, Y, R);
input [7:0] X, Y;
output [15:0] R;
    MULTIPLER4 #(.W(8)) L1 (.X(X), .Y(Y), .R(R));
endmodule
```

4.5.4.3基本库元件例化语句

- 在Verilog语言库中，已经对一些基本的门电路进行了定义，这些模块可以直接在程序中进行调用。
  - 多输入门：and、nand、or、nor、xor、xnor
  - 多输出门：buf、not
  - 三态门：bufif0、bufif1、notif0、notif1
  - 上拉、下拉电阻：pullup、pulldown
  - MOS开关：cmos、nmos、pmos、rcmos、rnmos、rpmos
  - 双向开关：tran、tranif0、tranif1、rtran、rtranif0、rtranif1

1. 多输入门

- 有多个输入端口，但只有一个输出端口。格式为：

基本门元件名 [例化名] （输出端口名，输入端口名1，输入端口名2，...）；

- 例化名可以省略；
- 输出端口默认放在端口关联表的第一个，后面是若干个输入端口。

例如：

```
and  U1 (dout, din1, din2);
or   U2 (dout, din1, din2, din3);
xor   (dout, din1, din2);
```

nand	0	1	x	z
0	1	1	1	1
1	1	0	x	x
x	1	x	x	x
z	1	x	x	x

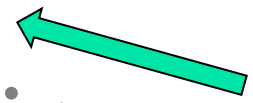
nor	0	1	x	z
0	1	0	x	x
1	0	0	0	0
x	x	0	x	x
z	x	0	x	x

图 5-11 多输入门 nand 和 nor 的逻辑处理方式↵

改写全加器程序的最后一个语句

```
module  F_Adder (a, b, cin, r, cout);
    input  a, b, cin;
    output  r, cout;
    wire  R1, C1, C2;

    H_Adder  U1 (.A(a),  .B(b),  .C(C1),  .R(R1));
    H_Adder  U2 (R1,  cin,  r,  C2);
    or  U3  (cout,  C1,  C2);
endmodule
```



assign cout = C1 | C2;

2. 多输出门

- 有多个输出端口，但只有一个输入端口。格式为：

基本门元件名 [例化名] （输出端口名1，…，输出端口名n，输入端口名）；

- 例化名可以省略；
- 输入端口默认放在端口关联表的最后一个，前面是若干个输出端口。

例如：

- **buf** B1 (dout1, dout2, dout3, din);
- **not** N1 (dout1, dout2, din);

buf	0	1	x	z
输出	0	1	x	x

not	0	1	x	z
输出	1	0	x	x

图 5-12　buf 和 not 的逻辑处理方式

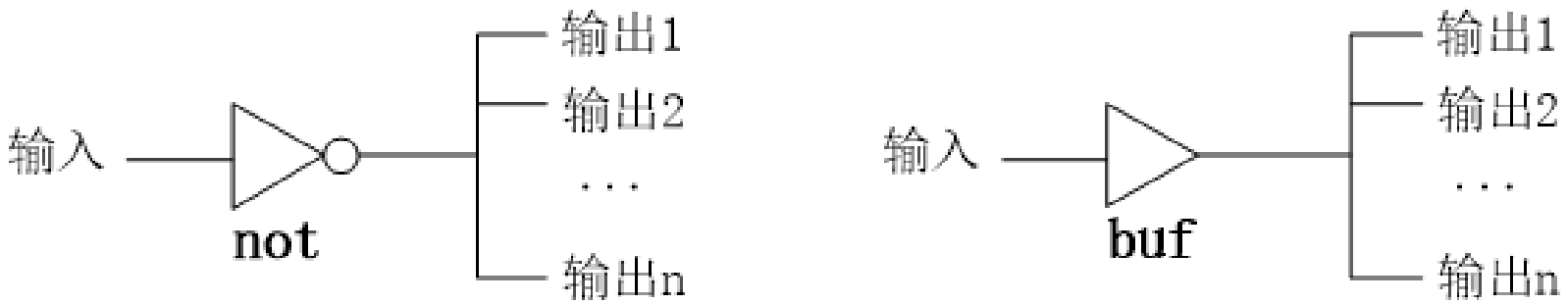


图 5-13　buf 和 not 的结构示意图

4.5.5 块语句

1. 顺序块语句

顺序块语句处于initial或always引导的过程语句中，用begin…end标识，语法格式为：

```
begin [：块名]
    [块内变量、参数定义];
    语句1;
    语句2;
     ……
    语句n;
end
```

- 可以给块语句取一个名字，也可以把块名省略掉；
- 在块语句内部可以根据需要定义块内变量或参数，它们只能在该块内使用，是局部量。
- 顺序块语句中的语句必须是顺序语句；

例如：

```
begin : SBLK
  reg [3:0] A;
  A = B & C;
  E = ~A;
end
```

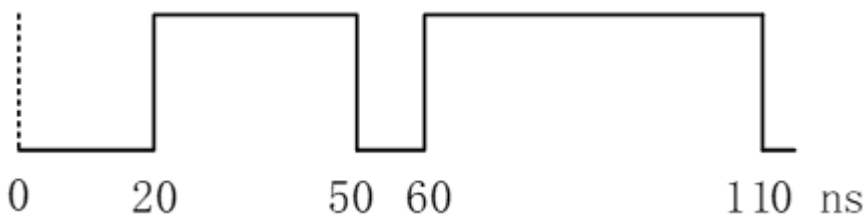
1. 并行块语句

并行块语句用**fork…join**标识，里面包含的语句为并行语句，语法格式为：

```
fork [：块名]
    [块内变量、参数定义];
    语句1;
    语句2;
     ……
    语句n;
join
```

- 可以给块语句取一个名字，也可以把块名省略掉；
- 在块语句内部可以根据需要定义块内变量或参数，它们只能在该块内使用，是局部量。
- 并行块语句中的语句是并行执行的，即便采用“=”符号；例如：

```
initial fork   //设时间单位为10ns
  t = 0;
  #2  t = 1;
  #5  t = 0;
  #6  t = 1;
  #11 t = 0;
join
```



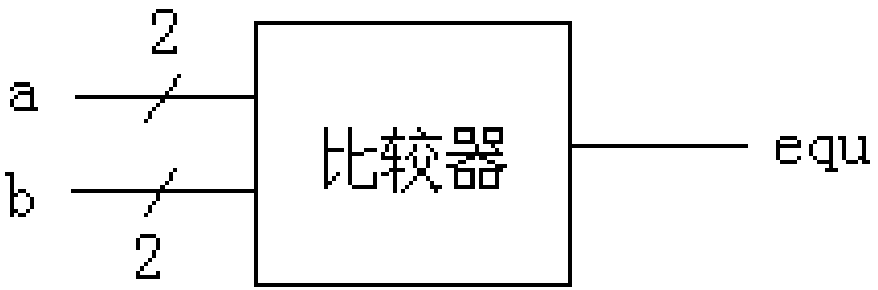
Verilog程序描述风格

- Verilog语句描述硬件电路的风格可以分为数据流描述、行为描述和结构化描述；
- 采用assign语句的描述方式为数据流描述方式；
- 采用always语句的描述方式为行为描述方式；
- 采用模块例化的描述方式为结构化描述方式。

描述风格实例

描述一个两位相等比较器，端口如图所示：

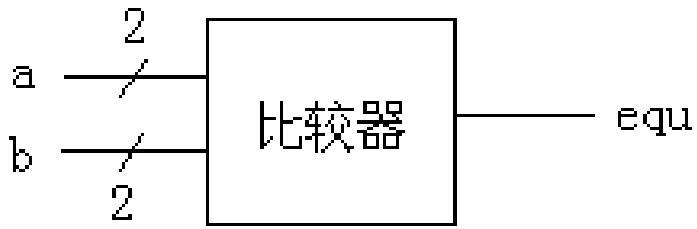
若a=b, 则equ=1；否则，equ=0。



1.数据流描述方式

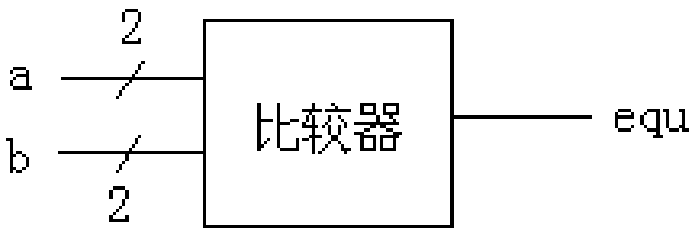
```
module  equ2  (a, b, equ) ;
    input  [1:0]  a,  b;
    output  equ;

    assign equ = (a= =b) ? 1 : 0;
endmodule
```



2.行为描述方式

```
module  equ2  (a, b, equ) ;
    input  [1:0]  a,  b;
    output  equ;
    reg equ;
    always @(a, b)
        if (a= =b)
            equ = 1’b1;
        else
            equ = 1’b0;
endmodule
```



3.结构化描述方式

```
module  equ2  (a, b, equ) ;
    input  [1:0]  a,  b;
    output  equ;
    wire e0, e1;
        xor  U1 (e0, a[0], b[0]);
        xor  U2 (e1, a[1], b[1]);
        nor  U3 (equ, e0, e1);
endmodule
```

